

Retail Analytics Engine

Advanced SQL Analytical Execution Matrix & Documentation

Target Schema: Pristine Star Schema
v1.0

Compatibility: PostgreSQL / Snowflake / BigQuery /
SQL Server

Classification: Technical Spec

1. Design Framework & Computational Objectives

This specification establishes the logical execution profiles for 16 core analytical questions targeted against the Retail Data Warehouse environment. These scripts map directly to the `fact_sales` layer and its related dimensions (`dim_product`, `dim_customer`, `dim_store`, `dim_date`).

To preserve raw computing efficiency, these patterns prioritize the use of standard **ANSI SQL window functions** (`DENSE_RANK()`, `SUM()`, `AVG()` with rolling frame windows, and `LAG()`) to maximize horizontal compute scaling while eliminating nested row loops or cross-joins.

2. Technical Execution Manifest & SQL Blueprints

Query 1: Top 10 High-Performing Products per Region per Fiscal Quarter

BUSINESS OBJECTIVE

Identifies core stock value driving primary product sales velocity categorized by structural operational geographies and quarterly seasonal timelines.

SQL REFERENCE PATTERN

```
SELECT region, quarter, year, product_name, total_revenue
FROM (
    SELECT s.region, d.quarter, d.year, p.product_name, SUM(f.total_amount -
f.discount_amount) as total_revenue,
        DENSE_RANK() OVER (PARTITION BY s.region, d.year, d.quarter ORDER BY
SUM(f.total_amount - f.discount_amount) DESC) as product_rank
    FROM fact_sales f
    JOIN dim_product p ON f.product_id = p.product_id
    JOIN dim_store s ON f.store_id = s.store_id
    JOIN dim_date d ON f.date_id = d.date_id
    GROUP BY s.region, d.year, d.quarter, p.product_name
) ranked_products
WHERE product_rank <= 10;
```

Query 2: Cumulative Rolling Year-to-Date (YTD) Revenue Profiles by Region

BUSINESS OBJECTIVE

Monitors dynamic compounding enterprise trajectory benchmarks across target physical bounds throughout the corporate financial year.

SQL REFERENCE PATTERN

```
SELECT s.region, d.full_date, d.year,
       SUM(f.total_amount - f.discount_amount) OVER (
         PARTITION BY s.region, d.year
         ORDER BY d.full_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS rolling_ytd_revenue
FROM fact_sales f
JOIN dim_store s ON f.store_id = s.store_id
JOIN dim_date d ON f.date_id = d.date_id;
```

Query 3: Year-over-Year (YoY) Sales Performance Deltas

BUSINESS OBJECTIVE

Evaluates baseline expansion growth trends by performing side-by-side matching of current metrics against matching seasonal profiles from past periods.

SQL REFERENCE PATTERN

```
WITH annual_sales AS (
  SELECT d.year, d.month, SUM(f.total_amount - f.discount_amount) AS monthly_revenue
  FROM fact_sales f
  JOIN dim_date d ON f.date_id = d.date_id
  GROUP BY d.year, d.month
)
SELECT current_year.year, current_year.month, current_year.monthly_revenue,
       prior_year.monthly_revenue AS prior_year_revenue,
       (current_year.monthly_revenue - prior_year.monthly_revenue) AS yoy_absolute_delta,
       ((current_year.monthly_revenue - prior_year.monthly_revenue) /
        prior_year.monthly_revenue) * 100 AS yoy_percentage_growth
FROM annual_sales current_year
LEFT JOIN annual_sales prior_year ON current_year.month = prior_year.month AND
current_year.year = prior_year.year + 1;
```

Query 4: Customer Lifetime Value (CLV) Global Ranking Matrices

BUSINESS OBJECTIVE

Identifies highly profitable customer cohorts across the global customer ecosystem to support precision marketing strategies.

```
SELECT c.customer_id, c.customer_name, c.segment,
       SUM(f.total_amount - f.discount_amount) AS clv_total_spend,
       DENSE_RANK() OVER (ORDER BY SUM(f.total_amount - f.discount_amount) DESC) AS
global_clv_rank
FROM fact_sales f
JOIN dim_customer c ON f.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name, c.segment;
```

Query 5: 7-Day Central Moving Average Rolling Windows

BUSINESS OBJECTIVE

Smooths out short-term operational transaction volatility to expose clear mid-week underlying retail demand signals.

SQL REFERENCE PATTERN

```
WITH daily_revenue AS (  
    SELECT d.full_date, SUM(f.total_amount - f.discount_amount) AS total_daily_sales  
    FROM fact_sales f  
    JOIN dim_date d ON f.date_id = d.date_id  
    GROUP BY d.full_date  
)  
SELECT full_date, total_daily_sales,  
    AVG(total_daily_sales) OVER (  
        ORDER BY full_date  
        ROWS BETWEEN 3 PRECEDING AND 3 FOLLOWING  
    ) AS centralized_7day_moving_average  
FROM daily_revenue;
```

Query 6: Prime Operational Store Generation by Region Mapping

BUSINESS OBJECTIVE

Identifies the single highest grossing storefront asset within each regional footprint to support cross-location performance benchmarking.

SQL REFERENCE PATTERN

```
SELECT region, store_name, total_sales  
FROM (  
    SELECT s.region, s.store_name, SUM(f.total_amount - f.discount_amount) AS total_sales,  
        ROW_NUMBER() OVER (PARTITION BY s.region ORDER BY SUM(f.total_amount -  
f.discount_amount) DESC) AS performance_rank  
    FROM fact_sales f  
    JOIN dim_store s ON f.store_id = s.store_id  
    GROUP BY s.region, s.store_name  
) ranked_stores  
WHERE performance_rank = 1;
```

Query 7: Month-over-Month (MoM) Financial Net Spikes

BUSINESS OBJECTIVE

Tracks short-term velocity changes to detect rapid shifts in operational momentum across consecutive fiscal periods.

SQL REFERENCE PATTERN

```
WITH monthly_metrics AS (  
    SELECT d.year, d.month, SUM(f.total_amount - f.discount_amount) AS revenue  
    FROM fact_sales f  
    JOIN dim_date d ON f.date_id = d.date_id  
    GROUP BY d.year, d.month  
)  
SELECT year, month, revenue,  
       LAG(revenue, 1) OVER (ORDER BY year, month) AS prior_month_revenue,  
       (revenue - LAG(revenue, 1) OVER (ORDER BY year, month)) AS net_mom_change,  
       ((revenue - LAG(revenue, 1) OVER (ORDER BY year, month)) / LAG(revenue, 1) OVER (ORDER  
BY year, month)) * 100 AS mom_percentage_velocity  
FROM monthly_metrics;
```

Query 8: Product Line Market Share Contribution Ratios within Categories

BUSINESS OBJECTIVE

Calculates the revenue contribution percentage of individual products relative to their parent categories to identify portfolio anchors.

SQL REFERENCE PATTERN

```
WITH item_revenues AS (  
    SELECT p.category, p.product_name, SUM(f.total_amount - f.discount_amount) AS item_rev  
    FROM fact_sales f  
    JOIN dim_product p ON f.product_id = p.product_id  
    GROUP BY p.category, p.product_name  
)  
SELECT category, product_name, item_rev,  
       SUM(item_rev) OVER (PARTITION BY category) AS total_category_revenue,  
       (item_rev / SUM(item_rev) OVER (PARTITION BY category)) * 100 AS  
dynamic_category_market_share_ratio  
FROM item_revenues;
```

Query 9: Inter-Purchase Temporal Variance Tracker (Days Since Prior Order)

BUSINESS OBJECTIVE

Measures consumer purchasing frequency and transaction intervals to flag churn risks or cooling engagement trends.

SQL REFERENCE PATTERN

```
SELECT f.customer_id, d.full_date AS current_order_date,
       LAG(d.full_date, 1) OVER (PARTITION BY f.customer_id ORDER BY d.full_date) AS
prior_order_date,
       (d.full_date - LAG(d.full_date, 1) OVER (PARTITION BY f.customer_id ORDER BY
d.full_date)) AS days_since_prior_order
FROM fact_sales f
JOIN dim_date d ON f.date_id = d.date_id;
```

Query 10: Cumulative Market Distribution Curve Trajectories

BUSINESS OBJECTIVE

Constructs Pareto or running distribution curves to visualize lifetime portfolio generation dynamics across time scales.

SQL REFERENCE PATTERN

```
WITH chronological_sales AS (
  SELECT d.full_date, SUM(f.total_amount - f.discount_amount) AS daily_yield
  FROM fact_sales f
  JOIN dim_date d ON f.date_id = d.date_id
  GROUP BY d.full_date
)
SELECT full_date, daily_yield,
       SUM(daily_yield) OVER (ORDER BY full_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS historical_cumulative_trajectory,
       SUM(daily_yield) OVER () AS global_aggregate_portfolio_yield,
       (SUM(daily_yield) OVER (ORDER BY full_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) / SUM(daily_yield) OVER ()) * 100 AS cumulative_percentage_trajectory_curve
FROM chronological_sales;
```

Query 11: Consumer Base Stratification via Spend Quartile Discretization

BUSINESS OBJECTIVE

Groups the customer base into four spend tiers (Quartiles 1-4) to optimize loyalty rewards and promotional alignment.

SQL REFERENCE PATTERN

```
WITH customer_spend AS (
  SELECT customer_id, SUM(total_amount - discount_amount) AS absolute_net_spend
  FROM fact_sales
  GROUP BY customer_id
)
SELECT customer_id, absolute_net_spend,
       NTILE(4) OVER (ORDER BY absolute_net_spend DESC) AS spend_quartile_bucket
FROM customer_spend;
```

Query 12: Chronological Customer Lifecycle Bounds (First vs. Most Recent Activity)

BUSINESS OBJECTIVE

Calculates individual customer lifecycle duration metrics to distinguish active accounts from long-term inactive buyers.

SQL REFERENCE PATTERN

```
SELECT c.customer_id, c.customer_name,  
       MIN(d.full_date) AS anchor_activation_date,  
       MAX(d.full_date) AS terminal_activity_date,  
       (MAX(d.full_date) - MIN(d.full_date)) AS absolute_lifecycle_duration_days  
FROM fact_sales f  
JOIN dim_customer c ON f.customer_id = c.customer_id  
JOIN dim_date d ON f.date_id = d.date_id  
GROUP BY c.customer_id, c.customer_name;
```

Query 13: Peak Intra-Month Transaction Thresholds Across Corporate Locations

BUSINESS OBJECTIVE

Identifies the exact calendar date that generated maximum sales volume for each store profile within a given fiscal month.

SQL REFERENCE PATTERN

```
SELECT store_id, store_name, year, month, full_date AS peak_transaction_date, daily_volume
FROM (
    SELECT s.store_id, s.store_name, d.year, d.month, d.full_date, SUM(f.total_amount -
    f.discount_amount) AS daily_volume,
        ROW_NUMBER() OVER (PARTITION BY s.store_id, d.year, d.month ORDER BY
    SUM(f.total_amount - f.discount_amount) DESC) AS sequence_rank
    FROM fact_sales f
    JOIN dim_store s ON f.store_id = s.store_id
    JOIN dim_date d ON f.date_id = d.date_id
    GROUP BY s.store_id, s.store_name, d.year, d.month, d.full_date
) sub_peak
WHERE sequence_rank = 1;
```

Query 14: Daily Execution Variance Analysis Over Contextual Monthly Midlines

BUSINESS OBJECTIVE

Measures daily performance deviations against the month's overall average to isolate mid-month demand spikes from baseline sales.

SQL REFERENCE PATTERN

```
WITH base_daily AS (
    SELECT d.year, d.month, d.full_date, SUM(f.total_amount - f.discount_amount) AS
    daily_revenue
    FROM fact_sales f
    JOIN dim_date d ON f.date_id = d.date_id
    GROUP BY d.year, d.month, d.full_date
)
SELECT year, month, full_date, daily_revenue,
    AVG(daily_revenue) OVER (PARTITION BY year, month) AS baseline_monthly_midline,
    (daily_revenue - AVG(daily_revenue) OVER (PARTITION BY year, month)) AS
    absolute_variance_from_midline
FROM base_daily;
```

Query 15: Top 3 Revenue-Driving Product Subcategories per Major Division

BUSINESS OBJECTIVE

Identifies the highest grossing product subcategories within each high-level category to guide inventory and display allocations.

SQL REFERENCE PATTERN

```
SELECT category, sub_category, subcat_revenue
FROM (
    SELECT p.category, p.sub_category, SUM(f.total_amount - f.discount_amount) AS
    subcat_revenue,
        DENSE_RANK() OVER (PARTITION BY p.category ORDER BY SUM(f.total_amount -
    f.discount_amount) DESC) AS rank_position
    FROM fact_sales f
    JOIN dim_product p ON f.product_id = p.product_id
    GROUP BY p.category, p.sub_category
) rank_subcats
WHERE rank_position <= 3;
```

Query 16: Trailing Linear Vector Velocity Flags (Consecutive Daily Spikes)

BUSINESS OBJECTIVE

Flags consecutive days of accelerating transaction growth to monitor operational pipeline constraints.

SQL REFERENCE PATTERN

```
WITH daily_timeline AS (
    SELECT d.full_date, SUM(f.total_amount - f.discount_amount) AS revenue
    FROM fact_sales f
    JOIN dim_date d ON f.date_id = d.date_id
    GROUP BY d.full_date
),
growth_vectors AS (
    SELECT full_date, revenue,
        LAG(revenue, 1) OVER (ORDER BY full_date) AS t_minus_1,
        LAG(revenue, 2) OVER (ORDER BY full_date) AS t_minus_2
    FROM daily_timeline
)
SELECT full_date, revenue, t_minus_1, t_minus_2,
    CASE
        WHEN revenue > t_minus_1 AND t_minus_1 > t_minus_2 THEN 'CRITICAL VECTOR: 2-DAY
ACCELERATION SPIKE'
        WHEN revenue > t_minus_1 THEN 'STABLE DIRECTIONAL ACCELERATION'
        ELSE 'FLAT OR DECELERATING VECTOR TRACK'
    END AS trailing_velocity_flag
FROM growth_vectors;
```